Reproducing “Near-Duplicate Detection for Web Crawling”

Name Surname

2026-08-01

Abstract This paper aims to reproduce the findings of the “Near-Duplicate Detection for Web Crawling” work, using PySpark to scale the proposed algorithm, under different datasets, feature models and parameters.

Introduction

Detecting duplicates in a dataset is crucial for tasks such as web crawling or AI model training, as it frees up computational resources. A straightforward approach to solve this involves hashing the data and comparing all rows via a Cartesian product. However, this approach is fragile: minor syntax differences in the data might produce entirely different hashes, which is undesirable. Moreover, the time complexity of a Cartesian product is $O(n^2)$, making it untractable for large datasets. The approach proposed by Manku et al. [1] suggest to identify near-duplicates instead. This method aims to generate similar hashes for similar data, leveraging this property to optimize the comparison algorithm. It’s also tested whether the algorithm is robust enough to find semantical near-duplicates, using a transformer-based embedding model. Lastly, some of the cited feature models and the proposed fingerprint matching algorithm are benchmarked against a variety of datasets, showcasing the efficiency of the suggested algorithm.

Methodology

The methodology proposed by Manku et al. [1] can be divided into a two-part pipeline.

1. Fingerprint Generation

The objective of this first phase is to transform text into a single, fixed-length binary value, referred to as a *fingerprint*. Specifically, similar data must corrispond to similar fingerprints (i.e. having a low Hamming distance). To achieve this, Charikar’s SimHash [2] is adopted: each text is converted into a set of fixed-length binary values called *features*, which are then assembled to form a fingerprint. A set of features is reduced into a fingerprint by selecting the most frequent bit value at each position: 1 if the majority of bits are 1 at that index, 0 otherwise.

Selecting the appropriate feature model and fingerprint size for a given dataset is important. The original paper proposes various feature models, such as *Shingles* and *TF-IDF*, ultimately selecting the latter with a fingerprint size of 64 bits. In this work, four feature models are tested and compared: *Bag of Words*, *Shingles*, *TF-IDF* and *Embeddings*, using fingerprint sizes ranging from 32 bits up to 512 bits.

Bag of Words

For a given text, its features are the individual words it contains. Hence, two texts are considered near-duplicate if they contain about the same words. This approach is computationally efficient and simple to implement, but it fails to preserve spatial locality. For example, the sentence "The bank is after the office" yields the exact same features as "The office is after the bank".

Shingles

Generally, a shingle is a contiguous sequence of n items from a data source; in this context, it refers to an overlapping contiguous sequence of n characters. Shingles are generated using a sliding window algorithm with a window size of n. This model preserves locality to some extent, but it generates a large number of features, which can negatively impact performance. To reduce the feature count, a non-overlapping variant is considered. Additionally, word-based shingles (as opposed to traditional character-based ones) are evaluated.

TF-IDF

This is a refined version of *Bag of Words* that accounts for both the frequency of a word within a specific text (DF) and its rarity across the entire dataset (IDF):

$$\text{TF-IDF}(term, document, Dataset) = TF(term, document) * IDF(term, Dataset)$$

TF stands for Term Frequency, which assigns higher importance to words that appear frequently in a given text:

$$TF(term, document) = \frac{\text{How many times term appears in document}}{\text{How many words document has}}$$

IDF stands for Inverse Document Frequency, which assigns higher weight to words that appear in fewer distinct documents. This filters out very common words, such as conjunctions and verbs:

$$IDF(term, Dataset) = \log(\frac{\text{Total number of documents in Dataset}}{\text{Number of documents that contain term}})$$

This is the feature model chosen in the original paper [1]. However, it is computationally more expensive than previous models, which were "local" in the sense that computing the features of a single text did not require information about the entire dataset. Furthermore, maintenance introduces additional overhead, as the weights of each term must be recomputed whenever the dataset changes.

Embeddings

An embedding model is a machine learning algorithm that maps text into a vector capturing its semantic meaning. In this study, a transformer-based model, *Qwen3-embedding-0.6b* [3], is used and executed locally. While all previous models rely on syntax to extract features, embedding models operate on semantics, allowing for the detection of semantically near-duplicate text.

2. Fingerprint Matching

Now every text in the dataset has an associated fingerprint. The goal for this phase is to compare fingerprints from two datasets, called *corpus* and *query* for simplicity sake, in order to find pairs of fingerprints which Hamming distance is less than K, a fixed integer value. The reference paper [1] approaches this problem by splitting each fingerprint into at least $K + n$ *partitions*, for $n > 0$. This grants a useful property: if two fingerprints are near-duplicate by a factor of K, then at least n of their

$K + n$ partitions must be equal (the opposite doesn't hold - other partitions might have more than K different bits). The optimal number of partitions is derived by the following formula:

$$X(f,k,d) = \begin{cases} 1 & \text{if } \ d < \tau \\ \min_{r>k}\binom{r}{k} \cdot X\left(\frac{fk}{r}, k, d - \frac{(r-k)f}{r}\right) & \text{otherwise} \end{cases}$$

where f is the size of each fingerprint, K is the maximum allowed Hamming distance, d is the log_2 cardinality of the dataset and $p_m in$ is the maximum allowed cardinality of the candidate set (in log_2).

Once the partitions are composed, Manku et al. [1] approach can be summerized as follows:
1. Each partition is sorted and compressed, using an Huffman code on the prefix of the fingerprints
2. Partitions are distributed to workers, so that partitions with matching schemas are sent to the same worker
3. Perform a binary search on all *query* rows, probing *corpus* - matching rows are sent back to the driver as candidates
4. The hamming distance of each candidate row is calculated, filtering rows whose distance is over *K*.
Unfortunately this algorithm cannot be implemented in PySpark, the chosen distributed framework. A variant, that makes full use of PySpark's capabilities, will be tracted in the following chapter.

Implementation
This implementation leverages the PySpark [4] framework, which allows to scale computing resources seamlessly.

Results
This is the results section.

Discussion
This is the discussion section.

Conclusion
This is the conclusion section.

Bibliography

[1] A. D. S. Gurmeet Singh Manku Arvind Jain, "Detecting Near-Duplicates for Web Crawling," 2007.

[2] M. S. Charikar, "Similarity Estimation Techniques from Rounding Algorithms," 2002.

[3] Y. Zhang *et al.*, "Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models," *arXiv preprint arXiv:2506.05176*, 2025.

[4] [Online]. Available: https://spark.apache.org/docs/latest/api/python/index.html